Chương 4 (Phần 2): Phát hiện lỗ hổng phần mềm – Phân tích động

4.4 Sinh đường thực thi – 4.5 Black-box Fuzzing – 4.6 White-box Fuzzing

ThS. Vũ Quang Dũng

Khoa Hệ thống thông tin – Trường Đại học Phenikaa

CSE703093 – An toàn phần mềm

Nhắc lại Phần 1

- 4.1: Security testing, fuzzing, oracle đơn giản (crash/không crash)
- 4.2: Code coverage, coverage-guided fuzzing
- 4.3: Execution tracing, taint tracking, magic byte problem

Phần 2 – **phần cuối cùng của học phần** – sẽ giải quyết vấn đề magic byte bằng white-box fuzzing (dùng lại SMT solving từ Chương 3), và tổng kết toàn bộ hành trình 4 chương.

Nội dung Phần 2

1. 4.4 Tạo ra các đường thực thi của chương trình
2. 4.5 Kiểm thử hộp đen (Black-box Fuzzing)
3. 4.6 Kiểm thử hộp trắng (White-box Fuzzing)

Nhắc lại Path Coverage và giới hạn (Chương 1)

- Path coverage (Chương 1, mục 1.4): bao phủ *mọi* đường thực thi có thể có – lý tưởng nhưng thường bất khả thi (path explosion)
- Mục 4.4 tập trung: làm sao **sinh ra một cách có hệ thống** các đường thực thi mới, thay vì chỉ dựa vào may rủi của input ngẫu nhiên (Phần 1)

Path Explosion – Nhắc lại và mở rộng (liên hệ State Explosion, Chương 1)

Định nghĩa

Path Explosion: số lượng đường thực thi có thể có tăng theo *cấp số mũ* với số điều kiện rẽ nhánh – một chương trình với n điều kiện độc lập có tới 2^n đường đi khác nhau.

Đây là phiên bản ”path-level” của *state explosion* đã học ở Chương 1 (Kripke structure) – cùng một vấn đề nền tảng xuất hiện ở nhiều mức trừu tượng khác nhau xuyên suốt học phần.

Ba chiến lược sinh đường thực thi

Random Walk	Chọn ngẫu nhiên nhánh tại mỗi điều kiện rẽ nhánh khi thực thi tượng trưng – đơn giản nhưng có thể bỏ sót đường quan trọng
DFS/BFS có hệ thống	Duyệt CFG (Chương 1) theo chiều sâu/rộng, đảm bảo không lặp lại đường đã xét
Coverage-guided (Best-first)	Ưu tiên khám phá đường có khả năng dẫn tới coverage mới cao nhất trước (đã học ở mục 4.2)

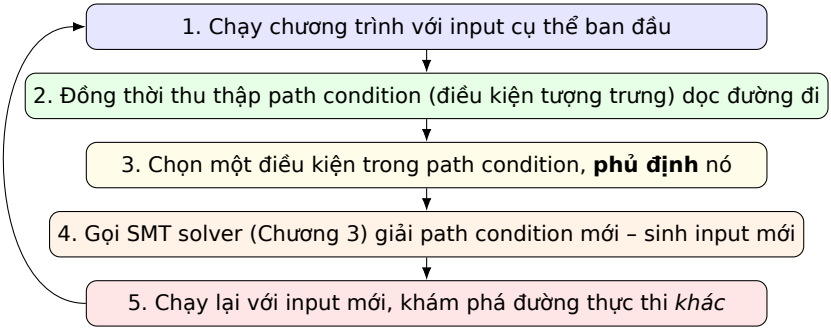
Concolic Execution – Khái niệm cốt lõi

Định nghĩa

Concolic Execution (concrete + symbolic): kỹ thuật chạy chương trình *đồng thời* với giá trị cụ thể (concrete) **và** giá trị tượng trưng (symbolic, đã học ở Chương 1 – symbolic execution), thu thập path condition dọc theo đường thực thi *cụ thể* đó.

Đây là cầu nối trực tiếp giữa dynamic testing (chạy thật) và static/symbolic analysis (Chương 1, 3) – nền tảng của white-box fuzzing sẽ học ở mục 4.6.

Quy trình Concolic Execution



Ví dụ Concolic Execution từng bước

```
void f(int x, int y) {
    if (x > 0) {            // Dieu kien 1
        if (y == x * 2) {  // Dieu kien 2
            crash();         // Muc tieu: den duoc day
        }
    }
}
```

Chạy với $x = 1, y = 0$: path condition = $(x > 0) \wedge (y \neq x \times 2)$. Phủ định điều kiện 2: $(x > 0) \wedge (y = x \times 2)$. SMT solver giải ra $x = 1, y = 2$ – input mới này sẽ đi vào nhánh `crash()`!

Vì sao Concolic tốt hơn Symbolic Execution thuần túy (Chương 1)

- Symbolic execution thuần túy (Chương 1) có thể gặp khó khi biểu thức quá phức tạp để SMT solver giải (vd: gọi hàm thư viện phức tạp, phép toán không tuyến tính)
- Concolic execution dùng giá trị **cụ thể** để ”vượt qua” các đoạn khó biểu diễn tượng trưng (concretize khi cần), chỉ giữ symbolic cho phần quan trọng
- Đánh đổi: mất một phần ”tính tổng quát” nhưng đạt được *khả năng mở rộng thực tế* tốt hơn nhiều

Xử lý Vòng lặp khi sinh đường thực thi

- Vòng lặp có thể tạo ra *vô hạn* đường thực thi khác nhau (mỗi số lần lặp khác nhau là một ”đường” khác)
- Kỹ thuật thực tế: giới hạn số lần ”mở” vòng lặp (loop bound), tương tự `--unwind` trong CBMC đã học ở Chương 3, mục 3.5
- Một số công cụ dùng *loop summarization*: tóm tắt hiệu ứng của toàn bộ vòng lặp bằng công thức thay vì mở từng bước – phức tạp hơn nhưng mở rộng tốt hơn

Search Strategy: Depth-First vs. Breadth-First

DFS	Đi sâu vào một đường đi cho tới cuối trước khi quay lại thử nhánh khác – tìm nhanh các lỗi ”sâu”, có thể bỏ sót lỗi nông ở nhánh khác lâu hơn
BFS	Xét mọi đường đi ở cùng độ sâu trước khi đi sâu hơn – đảm bảo không bỏ sót lỗi nông, nhưng chậm hơn để tìm lỗi sâu

Bài tập: Sinh đường thực thi cho hàm đơn giản

```
int classify(int x, int y) {
    if (x > 0) {
        if (y > 0) return 1;
        return 2;
    }
    return 3;
}
```

Với input ban đầu $x = 5, y = 5$ (đi vào nhánh `return 1`), hãy liệt kê các path condition cần phủ định để khám phá 2 nhánh còn lại bằng concolic execution.

Đáp án: Sinh đường thực thi

- Path ban đầu: $(x > 0) \wedge (y > 0)$ – dẫn tới `return 1`
- Phủ định điều kiện 2: $(x > 0) \wedge (y \leq 0)$ – SMT giải ra vd $x = 5, y = 0 \Rightarrow$ khám phá `return 2`
- Phủ định điều kiện 1: $(x \leq 0)$ – SMT giải ra vd $x = 0 \Rightarrow$ khám phá `return 3`
- Sau 2 bước, đạt 100% path coverage (3/3 nhánh) một cách có hệ thống, không cần thử ngẫu nhiên

Directed Symbolic Execution – Nhắm mục tiêu cụ thể

Định nghĩa

Directed Symbolic Execution: thay vì khám phá *mọi* đường thực thi, tập trung tìm đường dẫn tới một **vị trí mã nguồn cụ thể** (vd: dòng chứa cảnh báo static analysis từ Chương 2, 3).

Kết hợp trực tiếp với kết quả static analysis: nếu cppcheck (Chương 2) cảnh báo dòng 42 có thể lỗi, directed symbolic execution tìm cách sinh input *chính xác* đạt tới dòng 42 để xác nhận cảnh báo là thật hay false positive.

Test Case Generation vs. Bug Finding – Hai mục tiêu khác nhau

Test Case Generation	Mục tiêu: đạt coverage cao nhất có thể (mọi nhánh khả thi đều có ít nhất 1 test case)	KLEE (sẽ học ở mục
Bug Finding	Mục tiêu: tìm input gây crash/vi phạm sanitizer nhanh nhất, không cần bao phủ toàn bộ	

4.6) thiên về test case generation; nhiều fuzzer hybrid thiên về bug finding thuần túy – chiến lược tìm kiếm đường đi khác nhau tùy mục tiêu.

Path Prioritization – Ưu tiên đường nào trước?

- Với hàng nghìn đường thực thi tiềm năng, cần chiến lược ưu tiên thông minh hơn DFS/BFS thuần túy
- **Ưu tiên theo độ mới lạ**: đường đi qua nhiều block *ít được thăm* trước đó (tương tự coverage-guided, mục 4.2)
- **Ưu tiên theo độ gần với static warning**: đường có khả năng dẫn tới vị trí static analysis đã cảnh báo (directed symbolic execution)

Bài tập: So sánh DFS và Coverage-guided cho sinh đường thực thi

Với hàm có 10 điều kiện rẽ nhánh độc lập (tối đa $2^{10} = 1024$ đường), nếu chỉ có tài nguyên đủ thử 50 đường, chiến lược DFS thuần túy và coverage-guided sẽ khác nhau thế nào về kết quả?

Đáp án: So sánh DFS và Coverage-guided

- **DFS thuần túy**: có thể ”kẹt” khám phá sâu một nhánh con cụ thể (50 đường đều là biến thể của cùng một nhánh đầu tiên), bỏ sót hoàn toàn các nhánh khác
- **Coverage-guided**: ưu tiên đường mới về mặt *block đã thăm*, có xu hướng phân bổ đều hơn across toàn bộ 10 điều kiện, khám phá nhiều ”vùng” khác nhau của không gian trạng thái hơn với cùng ngân sách 50 lần thử

Tổng kết 4.4

- Path explosion là thách thức cốt lõi khi sinh đường thực thi, tương tự state explosion (Chương 1)
- Concolic execution kết hợp giá trị cụ thể và tượng trưng, dùng SMT (Chương 3) để sinh input mới có hệ thống
- Phủ định path condition từng bước cho phép khám phá nhánh mới một cách chủ động, không phụ thuộc may rủi
- Loop bound và search strategy (DFS/BFS) ảnh hưởng trực tiếp hiệu quả và phạm vi khám phá

Định nghĩa Black-box Fuzzing

Định nghĩa

Black-box Fuzzing: sinh input để thử chương trình mà **không cần biết** cấu trúc bên trong (mã nguồn, luồng điều khiển) – chỉ dựa trên input/output quan sát được từ bên ngoài.

Đối lập với white-box fuzzing (mục 4.6) – vốn tận dụng đầy đủ thông tin mã nguồn qua symbolic/concolic execution (mục 4.4).

Mutation-based Fuzzing – Ý tưởng cốt lõi

Định nghĩa

Mutation-based Fuzzing: bắt đầu từ input hợp lệ (seed, đã học ở mục 4.1), áp dụng các phép biến đổi nhỏ (đột biến) để tạo input mới.

Đây là chiến lược phổ biến nhất trong black-box fuzzing thực tế (AFL, libFuzzer mặc định) – đơn giản, hiệu quả, không cần hiểu cấu trúc định dạng input.

Các phép đột biến (Mutation Operators) cơ bản

Bit flip	Đảo ngược một hoặc vài bit ngẫu nhiên trong input
Byte flip	Thay đổi giá trị một byte ngẫu nhiên
Arithmetic	Cộng/trừ một số nhỏ vào giá trị số nguyên tìm thấy trong input
Block operations	Xóa, sao chép, hoặc chèn thêm một khối byte
Dictionary/Token	Chèn các ”từ khóa” đã biết quan trọng với định dạng (vd: `"PNG"`, `"<script>"`)

Ví dụ Bit Flip

```python
def bit_flip(data: bytearray, bit_pos: int) -> bytearray:
    data = bytearray(data)
    byte_idx = bit_pos // 8
    bit_idx = bit_pos % 8
    data[byte_idx] ^= (1 << bit_idx)   # XOR dao nguoc 1 bit
    return data
```

Phép đột biến đơn giản nhất, hiệu quả bất ngờ với các định dạng nhị phân có nhiều cờ (flag) đơn bit.

Ví dụ Arithmetic Mutation

```python
import struct

def arithmetic_mutate(data: bytearray, offset: int, delta: int) -> bytearray:
    data = bytearray(data)
    value = struct.unpack_from('<I', data, offset)[0]  # doc 4 byte little-endian
    new_value = (value + delta) % (2**32)
    struct.pack_into('<I', data, offset, new_value)
    return data
```

Rất hiệu quả để tìm lỗi **off-by-one** và **integer overflow** (Chương 2) – thử các giá trị ”gần” giá trị gốc như $-1, +1, +256$.

Havoc Mode – Chiến lược của AFL

Định nghĩa

Havoc: giai đoạn AFL áp dụng *nhiều phép đột biến ngẫu nhiên liên tiếp* lên cùng một input (kết hợp bit flip, byte flip, block operations...) trong một lần thử.

Sau khi các chiến lược có hệ thống (deterministic: bit flip tuần tự từng vị trí) đã ”cạn kiệt” ý tưởng mới, Havoc mode giúp khám phá các tổ hợp đột biến phức tạp hơn mà chiến lược tuần tự khó đạt tới.

Splicing – Kết hợp hai Input

Định nghĩa

Splicing: cắt và ghép (”lai tạo”) hai input khác nhau trong corpus để tạo input mới, kết hợp đặc điểm của cả hai.

Ví dụ

Input A (header hợp lệ + payload ngắn) ghép với Input B (header khác + payload dài) có thể tạo ra tổ hợp mới ”header A + payload B” chưa từng được thử riêng lẻ – một dạng ”lai giống” trong thuật toán tiến hóa (đã đề cập ở mục 4.2).

Generation-based Fuzzing – Khi biết cấu trúc định dạng

Định nghĩa

Generation-based Fuzzing: sinh input *từ đầu* dựa trên **văn phạm** (grammar) mô tả cấu trúc hợp lệ của định dạng input, thay vì đột biến từ seed có sẵn.

Hiệu quả hơn mutation-based với định dạng có cấu trúc phức tạp (JSON, XML, ngôn ngữ lập trình) – nhưng cần công sức viết văn phạm trước, và văn phạm không hoàn chỉnh có thể bỏ sót các trường hợp bất thường quan trọng.

Ví dụ Generation-based: Văn phạm đơn giản cho JSON

```python
import random

def generate_json_value(depth=0):
    if depth > 3:
        return random.choice(['null', '0', '"x"'])
    choice = random.choice(['object', 'array', 'number', 'string'])
    if choice == 'object':
        return '{"key": ' + generate_json_value(depth+1) + '}'
    elif choice == 'array':
        return '[' + generate_json_value(depth+1) + ']'
    elif choice == 'number':
        return str(random.randint(-1000, 1000))
    else:
        return '"' + 'x' * random.randint(0, 20) + '"'
```

So sánh Mutation-based vs. Generation-based

	Mutation-based	**Generation-based**
Cần seed corpus?	Có	Không bắt buộc
Cần hiểu định dạng?	Không	Có (viết văn phạm)
Hiệu quả với định dạng đơn giản	Rất tốt	Tốt nhưng dư thừa công sức
Hiệu quả với định dạng phức tạp	Kém (khó vượt qua magic byte)	Tốt hơn nhiều

Công cụ Black-box Fuzzing phổ biến

- **AFL/AFL++**: coverage-guided mutation-based fuzzer phổ biến nhất, mã nguồn mở
- **libFuzzer**: tích hợp trực tiếp vào chương trình qua một hàm ”harness” đơn giản, thường dùng chung với ASan (Chương 2)
- **Radamsa**: fuzzer tổng quát, tập trung vào việc sinh nhanh nhiều biến thể ”kỳ quặc” từ input mẫu
- **zzuf**: fuzzer đơn giản mức bit/byte flip, không có coverage feedback (gần giống fuzzing nguyên thủy của Barton Miller)

Ví dụ dùng libFuzzer – Viết Fuzzing Harness

```c
#include <stdint.h>
#include <stddef.h>

// Ham "harness": libFuzzer se goi ham nay lien tuc voi data ngau nhien
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    if (size < 4) return 0;
    parse_header(data, size);  // Ham muc tieu can fuzzing
    return 0;
}
```

```
clang -fsanitize=fuzzer,address harness.c -o fuzzer
./fuzzer corpus/
```

Ưu điểm của Black-box Fuzzing

- Đơn giản triển khai, không cần hiểu mã nguồn/logic nghiệp vụ
- Tốc độ thực thi cao (exec/s lớn, đã học ở mục 4.3) do không cần theo dõi chi tiết
- Áp dụng được cả khi *không có mã nguồn* (binary đóng gói, với DBI - Dynamic Binary Instrumentation)
- Đã chứng minh hiệu quả thực tế rộng rãi (OSS-Fuzz, mục 4.1)

Hạn chế của Black-box Fuzzing: Vấn đề Magic Byte (nhắc lại 4.3)

Lưu ý

Black-box fuzzing (kể cả coverage-guided) vẫn gặp khó khăn lớn với các điều kiện kiểm tra *giá trị cụ thể* (magic byte, checksum, chữ ký định dạng) – xác suất đột biến ngẫu nhiên ”trúng” đúng giá trị này gần như bằng 0. Đây chính là động lực cho **white-box fuzzing** (mục 4.6).

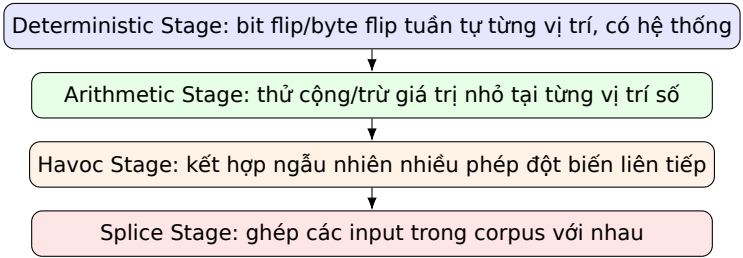
Bài tập: Thiết kế chiến lược Đột biến

Với một định dạng file có cấu trúc: 4 byte ”magic number” cố định + 4 byte độ dài + payload, hãy đề xuất tổ hợp mutation operators phù hợp để fuzzing hiệu quả (không cần văn phạm đầy đủ).

Đáp án: Thiết kế chiến lược Đột biến

- **Giữ nguyên** 4 byte magic number từ seed hợp lệ (không đột biến vùng này, hoặc dùng dictionary chèn đúng giá trị)
- **Arithmetic mutation** cho trường độ dài – thử các giá trị lệch (+1, -1, giá trị rất lớn) để tìm lỗi liên quan tới xử lý độ dài (Chương 2)
- **Bit/byte flip + block operations** tự do cho phần payload – đây là vùng nên khám phá rộng nhất

Deterministic vs. Havoc Stage trong AFL



AFL chạy các giai đoạn này tuần tự cho mỗi seed, cân bằng giữa khám phá có hệ thống và khám phá ngẫu nhiên sâu rộng.

Dictionary-based Fuzzing – Bổ sung Token quan trọng

Định nghĩa

Dictionary: danh sách các chuỗi/token quan trọng đối với định dạng input cụ thể (từ khóa ngôn ngữ, magic number, tên thẻ HTML...) được cung cấp thủ công cho fuzzer.

Ví dụ

Khi fuzzing một trình thông dịch JavaScript, dictionary có thể chứa: `"function"`, `"return"`, `"</script>"` – giúp fuzzer chèn đúng các token có ý nghĩa cú pháp thay vì phải ”phát minh lại” chúng qua đột biến ngẫu nhiên thuần túy.

Grammar-based Fuzzing nâng cao: Fuzzer chuyên dụng

- **Peach Fuzzer**: framework generation-based cho phép định nghĩa văn phạm phức tạp qua file cấu hình XML
- **Grammarinator**: sinh input tự động từ văn phạm ANTLR – phù hợp fuzzing trình biên dịch/thông dịch ngôn ngữ lập trình
- Với các định dạng phức tạp (HTML, SQL, ngôn ngữ lập trình), generation-based thường vượt trội mutation-based về khả năng tạo input ”có nghĩa cú pháp” mà vẫn đa dạng

Mutation-based Fuzzing bằng Python: Ví dụ hoàn chỉnh

```python
import random

def mutate(data: bytes) -> bytes:
    data = bytearray(data)
    op = random.choice(['bitflip', 'byteflip', 'insert', 'delete'])
    if op == 'bitflip' and data:
        idx = random.randrange(len(data))
        bit = random.randrange(8)
        data[idx] ^= (1 << bit)
    elif op == 'byteflip' and data:
        idx = random.randrange(len(data))
        data[idx] = random.randrange(256)
    elif op == 'insert':
        idx = random.randrange(len(data) + 1)
        data.insert(idx, random.randrange(256))
    elif op == 'delete' and len(data) > 1:
        idx = random.randrange(len(data))
        del data[idx]
    return bytes(data)
```

Regression Fuzzing – Fuzzing liên tục trong CI/CD

- Chạy fuzzing *ngắn hạn nhưng thường xuyên* (vd: 5 phút mỗi pull request) thay vì chỉ chạy dài hạn định kỳ
- Mục tiêu: bắt sớm các *regression* (lỗi mới do thay đổi mã nguồn gần đây gây ra), không nhất thiết tìm lỗi sâu chưa từng biết
- Corpus tích lũy qua thời gian (lưu trong repository) giúp mỗi lần fuzzing ngắn vẫn có điểm khởi đầu tốt, không phải bắt đầu lại từ đầu

Bài tập: Phân tích hiệu quả Dictionary

Một fuzzer black-box thuần túy (không dictionary) mất 10 giờ để tìm ra input hợp lệ bắt đầu bằng từ khóa `"BEGIN"` (5 ký tự). Nếu thêm dictionary chứa từ khóa này, thời gian dự kiến thay đổi thế nào? Giải thích bằng xác suất.

Đáp án: Phân tích hiệu quả Dictionary

- Không dictionary: xác suất đột biến ngẫu nhiên tạo đúng 5 ký tự ”BEGIN” liên tiếp là khoảng $1/256^5 \approx 1.4 \times 10^{-12}$ – cực kỳ hiếm
- Với dictionary: fuzzer có thể *chèn trực tiếp* token ”BEGIN” tại một vị trí ngẫu nhiên – xác suất thành công tăng lên nhiều bậc độ lớn, thời gian giảm từ hàng giờ xuống có thể chỉ vài giây/phút
- Đây là minh chứng định lượng cho giá trị thực tế của dictionary-based fuzzing

Tổng kết 4.5

- Black-box fuzzing không cần biết cấu trúc bên trong, dựa trên đột biến hoặc sinh theo văn phạm
- Mutation operators: bit/byte flip, arithmetic, block operations, dictionary/token
- Havoc mode và splicing giúp AFL khám phá tổ hợp phức tạp sau khi chiến lược tuần tự cạn kiệt
- Hạn chế cốt lõi: vấn đề magic byte – giải quyết bằng white-box fuzzing (mục 4.6)

Định nghĩa White-box Fuzzing

Định nghĩa

White-box Fuzzing: fuzzing có **đầy đủ thông tin** về cấu trúc bên trong chương trình (mã nguồn hoặc mã máy), sử dụng *concolic/symbolic execution* (mục 4.4, Chương 1) và *SMT solving* (Chương 3) để sinh input một cách có định hướng, thay vì đột biến ngẫu nhiên.

White-box Fuzzing giải quyết vấn đề Magic Byte thế nào

```
if (data[0]=='P' && data[1]=='N' && data[2]=='G') {
    parse_png(data);  // Kho voi black-box, de voi white-box!
}
```

White-box fuzzing thu thập path condition $(data[0]='P') \wedge (data[1]='N') \wedge (data[2]='G')$, đưa thẳng cho SMT solver (Chương 3) giải – solver trả về *chính xác* giá trị "PNG" cần thiết, không cần đoán ngẫu nhiên.

SAGE – White-box Fuzzer tiên phong (Microsoft)

- **SAGE** (Scalable Automated Guided Execution): công cụ white-box fuzzing đầu tiên ở quy mô công nghiệp (Microsoft Research, 2007)
- Dùng chiến lược ”Generational Search”: mỗi lần chạy, phủ định *lần lượt từng* điều kiện trong path condition để tối đa hóa số nhánh mới khám phá được từ một lần thực thi
- Đã phát hiện khoảng **1/3** tổng số lỗi tìm được bởi tất cả công cụ fuzzing dùng để kiểm thử Windows 7 trước khi phát hành

KLEE – Symbolic Execution Engine mã nguồn mở

- **KLEE**: công cụ symbolic execution phổ biến cho mã nguồn LLVM bitcode (C/C++)
- Tự động sinh test case đạt *high coverage* bằng cách khám phá có hệ thống mọi đường thực thi khả thi, dùng SMT solver (thường là STP hoặc Z3) nội bộ
- Khác biệt so với CBMC (Chương 3): KLEE tập trung vào *sinh test case với coverage cao*, CBMC tập trung vào *chứng minh/tìm phản ví dụ cho một thuộc tính cụ thể* trong phạm vi bounded

Constraint Solving Bottleneck – Thách thức chính của White-box Fuzzing

Lưu ý

Mỗi bước khám phá nhánh mới cần **gọi SMT solver** (Chương 3) – với chương trình lớn, số lượng path condition tăng nhanh, khiến solver trở thành ”nút thắt cổ chai” (bottleneck) làm giảm exec/s (mục 4.3) đáng kể so với black-box fuzzing thuần túy.

So sánh Black-box vs. White-box Fuzzing (Tổng hợp)

	Black-box	**White-box**
Cần mã nguồn?	Không bắt buộc	Cần (hoặc mã máy chi tiết)
Tốc độ (exec/s)	Rất cao	Thấp hơn nhiều (solver bottleneck)
Vượt qua magic byte?	Khó	Dễ (giải trực tiếp bằng SMT)
Mở rộng cho chương trình lớn?	Tốt	Hạn chế (path explosion, mục 4.4)

Hybrid Fuzzing – Kết hợp cả hai thế giới

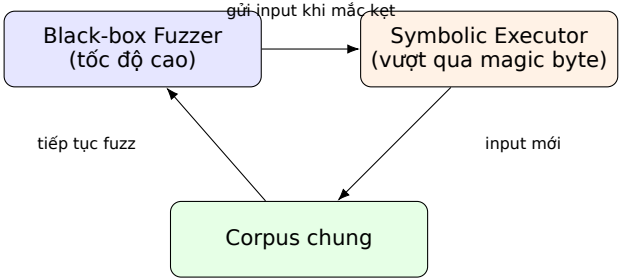
Định nghĩa

Hybrid Fuzzing: kết hợp black-box fuzzing (tốc độ cao, khám phá rộng) với white-box/concolic execution (chính xác, vượt qua magic byte) – chạy black-box làm chủ đạo, chỉ dùng white-box khi coverage ”mắc kẹt” (plateau, đã học ở mục 4.2).

Ví dụ

Driller (công cụ nghiên cứu nổi tiếng): chạy AFL (black-box) song song với symbolic execution có chọn lọc – chỉ kích hoạt symbolic execution khi AFL không tăng coverage sau một khoảng thời gian nhất định.

Kiến trúc Hybrid Fuzzing điển hình



Cài đặt White-box Fuzzing đơn giản bằng Z3 (Python)

```python
from z3 import Int, Solver, sat, Not, And

def negate_and_solve(path_conditions, index_to_negate):
    s = Solver()
    for i, cond in enumerate(path_conditions):
        s.add(Not(cond) if i == index_to_negate else cond)
    if s.check() == sat:
        return s.model()
    return None   # Nhanh nay khong the dat den (infeasible path)
```

Đây chính là hạt nhân của mọi công cụ white-box fuzzing: phủ định *từng* điều kiện trong path condition và giải lại bằng SMT.

Infeasible Path – Khi SMT trả về UNSAT

Định nghĩa

Infeasible Path: một đường thực thi *về mặt cú pháp có vẻ tồn tại* trong CFG (Chương 1) nhưng **không có input nào** thực sự kích hoạt được nó – SMT solver trả về UNSAT khi thử giải path condition tương ứng.

Ví dụ

`if (x > 10) { if (x < 5) { ... } }` – nhánh bên trong không bao giờ đạt được vì $x > 10 \wedge x < 5$ luôn UNSAT. White-box fuzzing *tự động phát hiện* và bỏ qua các nhánh này, tiết kiệm thời gian so với việc thử tìm input bằng đột biến ngẫu nhiên (vốn sẽ không bao giờ thành công).

Bài tập: Xác định Infeasible Path

```
void f(unsigned int x) {
    if (x > 100) {
        if (x < 0) {   // unsigned luon >= 0!
            crash();
        }
    }
}
```

Nhánh `crash()` có *feasible* (đạt được) hay không? Giải thích bằng kiến thức về kiểu `unsigned` đã học ở Chương 2.

Đáp án: Xác định Infeasible Path

- **Infeasible** – vì x là `unsigned int`, giá trị của nó **luôn** ≥ 0 (đã học ở Chương 2, mục 2.3 về kiểu số nguyên)
- Điều kiện $x < 0$ không bao giờ đúng với kiểu unsigned $\Rightarrow$ SMT solver (dùng bit-vector theory, Chương 3 mục 3.4) sẽ trả về UNSAT ngay lập tức khi thử giải
- Đây là ví dụ cụ thể minh họa lợi ích của việc mô hình hóa *chính xác kiểu dữ liệu* (bit-vector) thay vì số nguyên toán học thông thường

Bài tập: Thiết kế White-box Fuzzing cho hàm cụ thể

```
int check(int code) {
    if (code == 0x1337) {
        return 1;  // Muc tieu
    }
    return 0;
}
```

Hãy viết mã Python (Z3) để tự động sinh giá trị code đạt tới `return 1`, minh họa nguyên lý cốt lõi của white-box fuzzing.

Đáp án: Thiết kế White-box Fuzzing

```python
from z3 import Int, Solver, sat

code = Int('code')
s = Solver()
s.add(code == 0x1337)
if s.check() == sat:
    print(s.model())  # code = 4919 (= 0x1337)
```

Với black-box fuzzing thuần túy, xác suất đột biến ngẫu nhiên trúng đúng giá trị 0x1337 (trong không gian 2^{32} giá trị `int`) là gần như bằng 0 – minh chứng rõ ràng sức mạnh của white-box fuzzing.

Angr – Framework Symbolic Execution mã máy (Binary)

- **Angr**: framework Python phân tích và thực thi tượng trưng trực tiếp trên *mã máy nhị phân* (không cần mã nguồn), hỗ trợ nhiều kiến trúc CPU
- Dùng rộng rãi trong nghiên cứu bảo mật, phân tích malware, và các cuộc thi CTF (Capture The Flag)
- Là một trong những thành phần cốt lõi của Driller (hybrid fuzzer đã đề cập) – kết hợp Angr (symbolic) với AFL (black-box)

White-box Fuzzing cho Web Application (mở rộng)

- Nguyên lý white-box fuzzing không chỉ áp dụng cho C/binary – cũng áp dụng cho mã nguồn web (PHP, JavaScript, Python)
- Công cụ như `Codeql` phân tích mã nguồn theo hướng data-flow (liên hệ taint analysis, Chương 1-2) để tìm đường dẫn từ input người dùng tới sink nguy hiểm, tương tự nguyên lý concolic execution
- Ứng dụng: phát hiện SQL injection, XSS trong mã nguồn web bằng kỹ thuật tương tự đã học cho C

Chi phí triển khai White-box Fuzzing trong thực tế

Yêu cầu	Chi tiết
Mã nguồn/bitcode	Cần LLVM bitcode (KLEE) hoặc mã máy (Angr)
Mô hình hóa môi trường	Cần ”giả lập” input từ hệ điều hành, file, mạng dưới dạng symbolic
Xử lý thư viện ngoài	Hàm thư viện phức tạp (crypto, nén) thường cần ”concretize” thay vì symbolic hoàn toàn

Đây là lý do white-box fuzzing thường tốn công sức thiết lập ban đầu hơn nhiều so với chỉ chạy AFL trực tiếp.

Định nghĩa

Environment Modeling: mô hình hóa tương tác giữa chương trình và môi trường bên ngoài (đọc file, gọi hệ thống, thời gian) dưới dạng ký hiệu để symbolic execution có thể xử lý được.

Ví dụ

KLEE cung cấp phiên bản ”symbolic” của các hàm thư viện chuẩn C (`read`, `malloc`...) để giá trị trả về từ các lời gọi này cũng có thể là biến tượng trưng, không chỉ tham số đầu vào của hàm mục tiêu.

Merging Paths – Giảm bùng nổ đường thực thi trong Symbolic Execution

- Khi 2 nhánh hội tụ lại tại cùng một điểm trong CFG (Chương 1), có thể **gộp** (merge) các trạng thái tượng trưng lại thành một, thay vì tiếp tục theo dõi riêng biệt
- Giảm đáng kể số lượng ”trạng thái tượng trưng” cần duy trì đồng thời, giảm nhẹ vấn đề path explosion (mục 4.4)
- Đánh đổi: công thức path condition sau khi gộp trở nên phức tạp hơn (chứa mệnh đề ”nếu-thì-else”), có thể làm SMT solver chậm hơn khi giải

Bài tập: Phân tích chi phí Symbolic Execution

Một hàm có 20 điều kiện rẽ nhánh *liên tiếp không lồng nhau* (if-else if chuỗi dài). Ước lượng số lần gọi SMT solver cần thiết nếu dùng white-box fuzzing để đạt 100% branch coverage.

Đáp án: Phân tích chi phí Symbolic Execution

- Với chuỗi if-else if *không lồng nhau*, số nhánh chỉ tăng **tuyến tính** (20 nhánh, không phải 2^{20}) – mỗi điều kiện độc lập với các điều kiện khác
- Cần khoảng 20 lần gọi SMT solver (mỗi lần phủ định một điều kiện) để đạt 100% branch coverage – chi phí thấp hơn nhiều so với 10 điều kiện *lồng nhau* ở bài tập trước (tối đa $2^{10} = 1024$ tổ hợp)
- Bài học: **cấu trúc lồng nhau** (nested) mới thực sự gây bùng nổ tổ hợp, không phải chỉ số lượng điều kiện đơn thuần

Tổng kết 4.6

- White-box fuzzing dùng concolic/symbolic execution + SMT (Chương 1, 3) để sinh input có định hướng
- Giải quyết hiệu quả vấn đề magic byte mà black-box fuzzing khó vượt qua
- Constraint solving là bottleneck chính – hybrid fuzzing (Driller) kết hợp tốc độ black-box và độ chính xác white-box
- Infeasible path được tự động phát hiện qua UNSAT, tiết kiệm thời gian so với thử ngẫu nhiên vô ích

Bảng thuật ngữ cốt lõi Chương 4 (1/2)

Concolic Execution	Kết hợp thực thi cụ thể (concrete) và tượng trưng (symbolic)
Path Explosion	Số đường thực thi tăng theo cấp số mũ với số điều kiện rẽ nhánh
Mutation-based Fuzzing	Đột biến từ seed có sẵn (bit/byte flip, arithmetic, block ops)
Generation-based Fuzzing	Sinh input từ đầu theo văn phạm định dạng
Havoc Mode	Kết hợp ngẫu nhiên nhiều đột biến liên tiếp (AFL)

Bảng thuật ngữ cốt lõi Chương 4 (2/2)

White-box Fuzzing	Dùng symbolic execution + SMT để sinh input có định hướng
Infeasible Path	Đường thực thi không có input nào kích hoạt được (SMT trả UNSAT)
Hybrid Fuzzing	Kết hợp black-box (tốc độ) và white-box (chính xác)
SAGE/KLEE/Angr	Các công cụ white-box fuzzing/symbolic execution tiêu biểu
Environment Modeling	Mô hình hóa tương tác với hệ điều hành dưới dạng symbolic

Câu hỏi ôn tập tổng hợp Phần 2 (1/3)

1. Vì sao Concolic Execution thực tế hơn Symbolic Execution thuần túy?
2. Phân biệt Mutation-based và Generation-based fuzzing, cho ví dụ mỗi loại
3. Vì sao White-box fuzzing giải quyết được vấn đề magic byte mà Black-box khó làm được?

Câu hỏi ôn tập tổng hợp Phần 2 (2/3)

4. Constraint solving bottleneck trong White-box fuzzing là gì?
5. Hybrid Fuzzing (Driller) hoạt động theo nguyên lý gì?
6. Infeasible path được phát hiện tự động như thế nào?

Câu hỏi ôn tập tổng hợp Phần 2 (3/3)

7. Vì sao cấu trúc lồng nhau (nested) gây bùng nổ tổ hợp nhiều hơn chuỗi if-else if không lồng?
8. Dictionary-based fuzzing cải thiện hiệu quả thế nào so với đột biến ngẫu nhiên thuần túy?
9. Path Merging trong symbolic execution đánh đổi điều gì?

Đáp án gợi ý câu 1–3

1. Vì Concolic dùng giá trị cụ thể để ”vượt qua” phần khó biểu diễn tượng trưng (hàm thư viện phức tạp), chỉ giữ symbolic cho phần cần thiết – mở rộng tốt hơn symbolic thuần túy
2. Mutation: đột biến ảnh PNG có sẵn (AFL mặc định). Generation: sinh câu JSON hợp lệ từ văn phạm (Peach Fuzzer, Grammarinator)
3. Vì White-box thu thập path condition chính xác (vd: $data[0] =$’P’) và dùng SMT giải trực tiếp ra giá trị cần thiết, không cần đoán ngẫu nhiên như Black-box

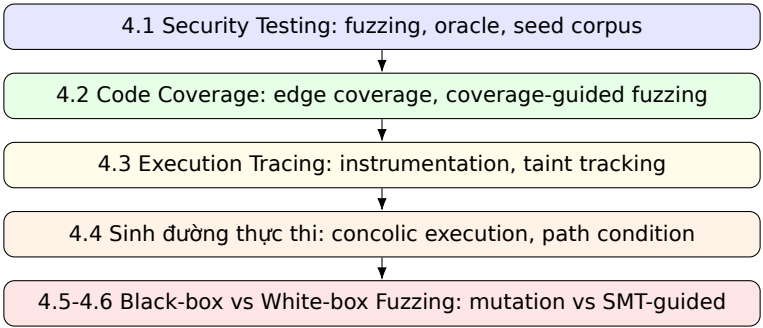
Đáp án gợi ý câu 4–6

4. Mỗi bước khám phá nhánh mới cần gọi SMT solver – với chương trình lớn, số lượng lời gọi tăng nhanh, làm chậm exec/s đáng kể so với black-box thuần túy
5. Chạy black-box (AFL) làm chủ đạo tốc độ cao; khi coverage chững lại (plateau), kích hoạt symbolic execution có chọn lọc để vượt qua điều kiện khó, rồi quay lại black-box
6. Khi SMT solver trả về UNSAT cho path condition tương ứng với đường đó – xác nhận hình thức rằng không tồn tại input nào kích hoạt được

Đáp án gợi ý câu 7–9

7. Vì điều kiện lồng nhau tạo ra *tổ hợp* giữa các điều kiện (mỗi điều kiện con phụ thuộc điều kiện cha), trong khi chuỗi độc lập chỉ cộng dồn tuyến tính, không nhân lên
8. Dictionary cho phép chèn trực tiếp token có ý nghĩa cú pháp (như ”BEGIN”) thay vì phải ”phát minh lại” qua đột biến ngẫu nhiên – tăng xác suất thành công lên nhiều bậc độ lớn
9. Path Merging giảm số trạng thái tượng trưng cần duy trì (giảm path explosion) nhưng làm công thức path condition phức tạp hơn, có thể làm chậm SMT solver khi giải

Bản đồ tri thức Chương 4



Luyện tập trắc nghiệm – Câu 1–3

Câu 1. Concolic Execution là sự kết hợp của những gì?
A. Static và Dynamic Analysis B. Concrete và Symbolic Execution C. Black-box và White-box D. SAT và SMT
Câu 2. Mutation-based fuzzing khác Generation-based fuzzing ở điểm nào?
A. Mutation cần văn phạm, Generation thì không B. Mutation đột biến từ seed có sẵn, Generation sinh từ đầu theo văn phạm C. Không có khác biệt D. Generation luôn nhanh hơn
Câu 3. Vấn đề magic byte được giải quyết hiệu quả nhất bằng kỹ thuật nào?
A. Bit flip B. Havoc mode C. White-box fuzzing với SMT solver D. Splicing

Đáp án Câu 1–3

1. B (Concrete và Symbolic Execution).
2. B (Mutation đột biến từ seed, Generation sinh từ đầu theo văn phạm).
3. C (White-box fuzzing với SMT solver).

Luyện tập trắc nghiệm – Câu 4–6

Câu 4. Công cụ SAGE (Microsoft) sử dụng chiến lược gì?
A. Random mutation B. Generational Search – phủ định lần lượt từng điều kiện C. Chỉ dùng grammar D. Chỉ chạy trên Linux
Câu 5. Hybrid Fuzzing (vd: Driller) kết hợp điều gì?
A. Hai fuzzer black-box khác nhau B. Black-box fuzzing tốc độ cao + symbolic execution khi cần C. Hai SMT solver khác nhau D. Static analysis và code review
Câu 6. Infeasible path được phát hiện khi nào?
A. Khi chương trình crash B. Khi SMT solver trả về UNSAT cho path condition C. Khi coverage đạt 100% D. Khi test case chạy quá lâu

Đáp án Câu 4–6

4. B (Generational Search).
5. B (Black-box tốc độ cao + symbolic execution khi cần, vd Driller).
6. B (Khi SMT solver trả về UNSAT).

Bài tập tổng hợp cuối chương

Một đội ngũ fuzzing một thư viện phân tích file nén (compression) trong 1 tuần bằng AFL (black-box), coverage đạt 65% rồi chững lại (plateau). Đề xuất bước tiếp theo dựa trên toàn bộ kiến thức Chương 4.

Gợi ý đáp án bài tập tổng hợp

1. Kiểm tra xem có **magic byte/checksum** nào chặn đường vào các phần chưa được coverage (mục 4.3, 4.5) – định dạng nén thường có magic number và checksum
2. Nếu đúng, chuyển sang **hybrid fuzzing** (Driller-style): dùng symbolic execution có chọn lọc để vượt qua các điều kiện đó
3. Sau khi vượt qua, quay lại AFL tốc độ cao để tiếp tục khám phá phần mới với corpus đã được ”làm giàu”
4. Định kỳ chạy corpus distillation (mục 4.2) để giữ hiệu năng fuzzing ổn định

Hoàn thiện dự án Python: module ch4_dynafuzz

Module ch4_dynafuzz (cuối cùng của softsec-toolkit) gồm:

- coverage_tracker.py: theo dõi coverage (dòng/nhánh đã thực thi) cho hàm Python mục tiêu
- blackbox_fuzzer.py: fuzzer đột biến (bit flip, byte flip, arithmetic) với coverage feedback
- whitebox_fuzzer.py: concolic fuzzer dùng Z3 (kết hợp module ch3_staticsat) để phủ định path condition và sinh input vượt qua magic byte

Demo module: blackbox_fuzzer

```
from softsec.ch4_dynafuzz import blackbox_fuzz

def target(data: bytes) -> bool:
    if len(data) >= 4 and data[:4] == b"FUZZ":
        return True  # "crash" gia lap
    return False

result = blackbox_fuzz(target, seed=b"AAAA", iterations=5000)
print(result.found_crash, result.crashing_input)
```

Demo module: whitebox_fuzzer

```python
from softsec.ch4_dynafuzz import whitebox_fuzz

def target_symbolic(x):
    if x == 0x1337:
        return "CRASH"
    return "OK"

result = whitebox_fuzz(target_symbolic, num_vars=1, target_branch="CRASH")
print(result.solution)  # {'x': 4919}
```

Fuzzer trắng tìm ra giá trị chính xác ngay lập tức nhờ SMT, trong khi fuzzer đen (bài tập trên) có thể cần rất nhiều lần thử để ”may mắn” trúng magic value tương tự.

Checklist tự đánh giá cuối Chương 4

- ☐ Tôi hiểu sự khác biệt giữa black-box và white-box fuzzing, và khi nào nên dùng mỗi loại
- ☐ Tôi có thể giải thích vấn đề magic byte và cách concolic execution giải quyết nó
- ☐ Tôi hiểu vai trò của coverage feedback trong việc hướng dẫn fuzzer
- ☐ Tôi có thể viết một fuzzer đơn giản (mutation-based) và một concolic fuzzer cơ bản bằng Z3

Danh sách kỹ năng thực hành đã đạt được (Chương 4)

Sau Chương 4, sinh viên đã có thể:

- Viết một mutation-based fuzzer đơn giản bằng Python với các phép đột biến cơ bản
- Giải thích và minh họa vấn đề magic byte bằng ví dụ cụ thể
- Viết ràng buộc Z3 để giải path condition và sinh input vượt qua điều kiện khó (white-box fuzzing cơ bản)
- Thiết kế chiến lược fuzzing phù hợp (black-box, white-box, hay hybrid) theo đặc điểm chương trình mục tiêu

So sánh nhanh: Chương 3 vs Chương 4

	Chương 3 (Static)	**Chương 4 (Dynamic)**	
Cách tiếp cận	Chứng minh hình thức	Thực thi thử nghiệm	
Độ tin cậy	Cao (sound trong phạm vi)	Trung bình (under-approx)	
Chi phí	Cao (SMT solving)	Thấp hơn (đặc biệt black-box)	Cả hai bổ sung lẫn
Điểm mạnh	Không bỏ sót lỗi trong phạm vi	Phát hiện lỗi thực tế nhanh, dễ mở rộng	

nhau – đây chính là lý do hybrid fuzzing (kết hợp cả hai) là hướng đi thực tế phổ biến nhất hiện nay.

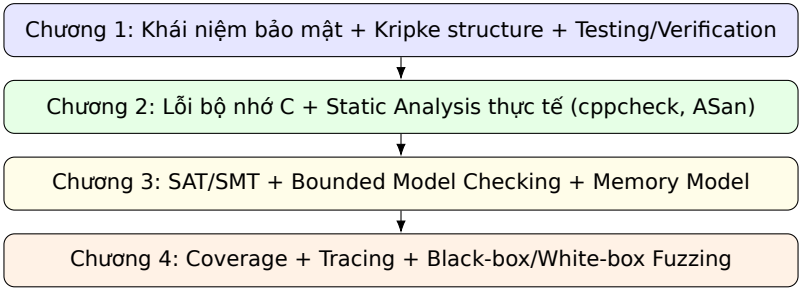
Điểm cần lưu ý khi ôn tập Chương 4

- Nắm chắc sự khác biệt cốt lõi: Black-box (nhanh, không cần mã nguồn) vs White-box (chính xác, cần mã nguồn + SMT)
- Hiểu rõ *tại sao* magic byte khó với black-box và *cách* white-box giải quyết bằng SMT
- Ôn lại toàn bộ 6 câu trắc nghiệm có đáp án ở cả 2 phần của Chương 4, đặc biệt các khái niệm liên hệ ngược lại Chương 1-3

Hỏi đáp

Câu hỏi?

Tổng kết toàn bộ học phần CSE703093



Toàn bộ 4 chương hình thành một **vòng khép kín**: mô hình hóa (1) $\rightarrow$ lỗi cụ thể (2) $\rightarrow$ chứng minh hình thức (3) $\rightarrow$ kiểm thử thực nghiệm (4) – softsec-toolkit tích hợp cả bốn thành một bộ công cụ liền mạch.

Bảng tổng hợp toàn bộ Sound/Complete xuyên suốt học phần

Kỹ thuật	**Sound?**	**Complete?**	**Chương**
Testing thủ công	Không	Không	1
Static Analysis (pattern)	Không hoàn toàn	Không hoàn toàn	2
Abstract Interpretation	Có (over-approx)	Không	2, 3
BMC (dưới CT)	Có (trong k)	Không	3
BMC (đạt CT)	Có	Có	3
Black-box Fuzzing	Không	Không	4
White-box Fuzzing	Không (under-approx)	Không (nhưng crash luôn thật)	4

Softsec-toolkit: Bản đồ đầy đủ 4 module

Module	Chương	Thành phần chính
ch1_models	1	KripkeStructure, test_design, risk
ch2_memsafe	2	pattern_scanner, memory_trace, struct_layout
ch3_staticsat	3	sat_solver, smt_wrapper, bounded_checker
ch4_dynafuzz	4	coverage_tracker, blackbox_fuzzer, whitebox_fuzzer

Bốn module này cùng nhau tạo thành một bộ công cụ hoàn chỉnh, mỗi module xây dựng dựa trên module trước – phản ánh chính cấu trúc logic của toàn bộ học phần.

Lời khuyên cuối cùng cho Bài tập lớn

- Chọn một chương trình/hệ thống **đủ nhỏ để hiểu sâu**, không cần quá lớn hay phức tạp
- Áp dụng *đầy đủ quy trình* 4 chương: mô hình hóa $\rightarrow$ tìm lỗi cụ thể $\rightarrow$ kiểm chứng hình thức (nếu phù hợp) $\rightarrow$ fuzzing xác nhận
- Tận dụng tối đa `softsec-toolkit` đã xây dựng xuyên suốt học phần thay vì viết lại từ đầu

Chuẩn bị cho Bài tập lớn cuối kỳ

- Bài tập lớn (KTHP, 50% điểm) yêu cầu áp dụng *tổng hợp* kiến thức 4 chương lên một hệ thống/chương trình do sinh viên tự chọn
- Xem chi tiết hướng dẫn đề tài và hướng dẫn báo cáo trong tài liệu riêng của học phần
- Toàn bộ module `softsec-toolkit` (`ch1_models` đến `ch4_dynafuzz`) là nền tảng công cụ sẵn sàng sử dụng cho bài tập lớn

Lời cảm ơn

Cảm ơn các bạn sinh viên đã đồng hành cùng học phần CSE703093 – An toàn phần mềm qua 4 chương và hàng trăm bài tập thực hành. Hy vọng bộ công cụ `softsec-toolkit` sẽ tiếp tục hữu ích cho các bạn ngoài phạm vi học phần này.

Danh sách tài liệu tham khảo tổng hợp

- Graham, van Veenendaal, Evans, Black – *Foundations of Software Testing* (giáo trình chính)
- Clarke, Grumberg, Peled – *Model Checking*
- Kroening, Strichman – *Decision Procedures*
- Sutton, Greene, Amini – *Fuzzing: Brute Force Vulnerability Discovery*
- Tài liệu chính thức: CERT C, MISRA C, CBMC, Z3, AFL++, KLEE

Con đường học tập tiếp theo (gợi ý)

- Tham gia các cuộc thi CTF (Capture The Flag) để thực hành kỹ năng phân tích lỗ hổng thực tế
- Đóng góp vào OSS-Fuzz hoặc dự án fuzzing mã nguồn mở khác
- Tìm hiểu sâu hơn về Rust (an toàn bộ nhớ tại compile-time) như một hướng bổ sung cho C truyền thống
- Khám phá các công cụ nâng cao: KLEE, Angr, CBMC ở mức thực hành chuyên sâu hơn

Câu hỏi cuối cùng để tự suy ngẫm

Nhìn lại toàn bộ 4 chương, khái niệm hoặc kỹ thuật nào bạn thấy có khả năng ứng dụng cao nhất trong công việc tương lai của mình? Hãy dành vài phút ghi chú lại trước khi chuyển sang bài tập lớn.

Điểm cần lưu ý cuối cùng trước khi thi và làm bài tập lớn

- Kiểm tra giữa kỳ (trắc nghiệm) bao phủ Chương 1 và 2 – ôn lại kỹ các công thức và phân loại đã học
- Bài tập lớn (không thuyết trình) bao phủ toàn bộ 4 chương – ưu tiên chọn đề tài phù hợp với năng lực và thời gian thực tế
- Tận dụng `softsec-toolkit` và tài liệu tham khảo đã được cung cấp xuyên suốt học phần

Lời kết

Hoàn thành CSE703093 – An toàn phần mềm

Chúc các bạn áp dụng thành công kiến thức đã học vào bài tập lớn và thực tế công việc!